Dokumentasi Praktikum Pemrograman Mobile

Pertemuan 5



Nama:

Muhammad Yusuf Bagus Syafaat

NIM:
20240801041

UNIVERSITAS ESA UNGGUL TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

2026

1. Pendahuluan

Praktikum Pertemuan 5 membahas penerapan penyimpanan lokal pada aplikasi Flutter. Pembahasan mencakup penyimpanan data sederhana menggunakan SharedPreferences dan penyimpanan data terstruktur menggunakan SQLite melalui package sqflite. Implementasi kemudian dikembangkan menjadi aplikasi catatan dengan operasi CRUD, pencarian, pengurutan, migrasi database, serta aplikasi Pencatat Pengeluaran.

2. Tujuan Praktikum

- Memahami konsep persistence atau penyimpanan data lokal pada aplikasi Flutter.
- Menggunakan SharedPreferences untuk menyimpan data sederhana berbentuk key-value.
- Membuat database lokal menggunakan SQLite dengan package sqflite dan path.
- Menerapkan operasi Create, Read, Update, dan Delete (CRUD).
- Menerapkan pencarian, pengurutan, validasi, dan migrasi database.
- Membangun aplikasi Pencatat Pengeluaran dengan penyimpanan lokal.

3. Dasar Teori

3.1 Penyimpanan Lokal

Data yang hanya berada pada state aplikasi dapat hilang ketika aplikasi dihentikan. Persistence digunakan agar data tetap tersedia pada perangkat setelah aplikasi dijalankan kembali.

3.2 SharedPreferences

SharedPreferences merupakan penyimpanan key-value yang sesuai untuk data sederhana seperti nama pengguna, pilihan tema, penghitung pembukaan aplikasi, atau preferensi urutan.

Metode	Kegunaan
setString / getString	Menyimpan dan membaca teks
setBool / getBool	Menyimpan dan membaca nilai true/false
setInt / getInt	Menyimpan dan membaca bilangan bulat

3.3 SQLite dan sqflite

SQLite digunakan untuk data terstruktur yang terdiri atas tabel dan kolom. Package sqflite menyediakan akses database SQLite dari Flutter, sedangkan package path digunakan untuk membentuk lokasi file database.

3.4 CRUD

Operasi	SQL	Implementasi
Create	INSERT	db.insert()
Read	SELECT	db.query()
Update	UPDATE	db.update()
Delete	DELETE	db.delete()

4. Alat dan Bahan

- Flutter SDK
- Dart
- Visual Studio Code atau Android Studio
- Emulator/perangkat Android
- shared_preferences
- sqflite
- path

5. Implementasi Praktikum

5.1 Bagian A : SharedPreferences

Bagian A membuat halaman Pengaturan untuk menyimpan nama, mode gelap, dan jumlah pembukaan aplikasi. Kode menggunakan SharedPreferences dan memuat data kembali pada initState. Pasang paket dengan perintah *flutter pub add shared_preferences* terlebih dahulu.

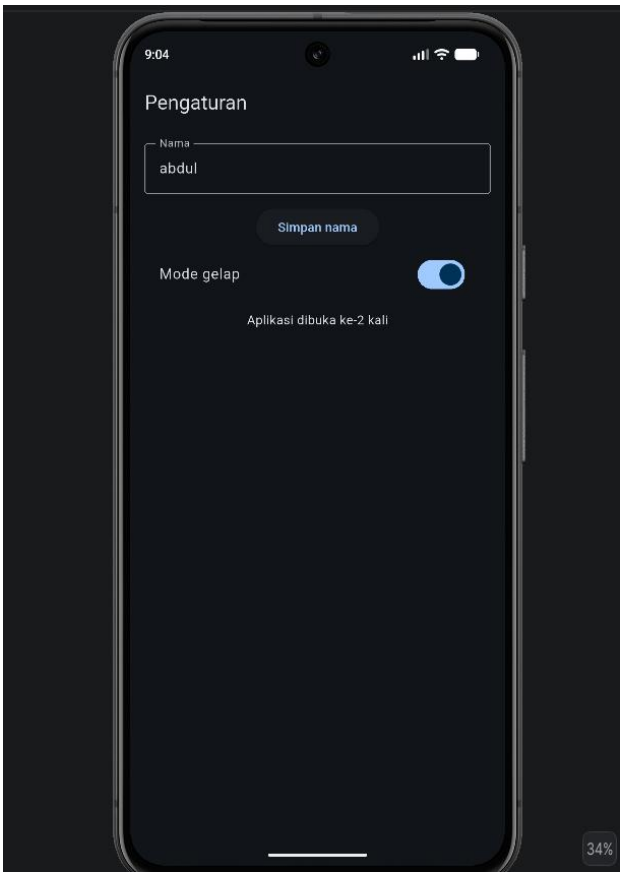
```
PS C:\Users\LENOVO LOQ\pemmob\pert5\praktikum5> flutter pub add shared_preferences
Resolving dependencies...
Downloading packages... (2.9s)
  cupertino_icons 1.0.9 (2.0.0 available)
+ ffi 2.2.0
+ file 7.0.1
+ flutter_web_plugins 0.0.0 from sdk flutter
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
+ path_provider_linux 2.2.2
+ path_provider_platform_interface 2.1.3
+ path_provider_windows 2.3.0
```

```
final prefs = await SharedPreferences.getInstance();

_controller.text = prefs.getString('nama') ?? '';
_gelap = prefs.getBool('gelap') ?? false;
_bukaKe = (prefs.getInt('bukaKe') ?? 0) + 1;

await prefs.setString('nama', _controller.text.trim());
await prefs.setBool('gelap', nilai);
await prefs.setInt('bukaKe', _bukaKe);
```

Implementasi tersebut menunjukkan bahwa nilai disimpan menggunakan pasangan key-value dan dibaca kembali ketika halaman dibuat. Pada kode yang digunakan, key yang dipakai adalah nama, gelap, dan bukaKe. Dengan output:



5.2 Bagian B : Menyiapkan SQLite

Pertama, tahapnya adalah membuat proyek baru lalu meinstall sqlite path di terminal dengan perintah *flutter pub add sqflite path*

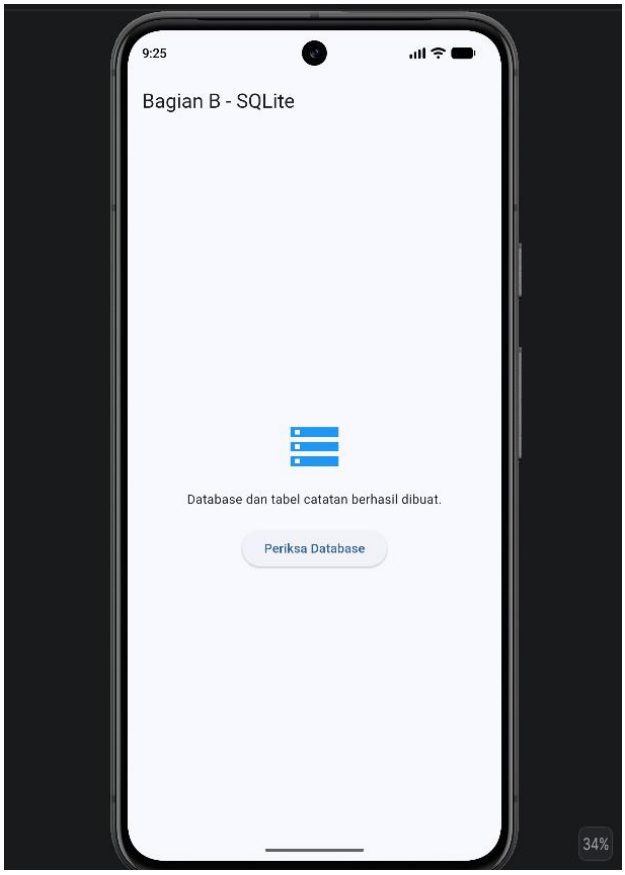
```
PS C:\Users\LENOVO LOQ\pemmob\pert5\praktikum5> flutter pub add sqflite path
Resolving dependencies...
Downloading packages...
  cupertino_icons 1.0.9 (2.0.0 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.18.3 (1.19.0 available)
  path 1.9.1 (from transitive dependency to direct dependency)
+ sqflite 2.4.4+1
+ sqflite_android 2.4.4
+ sqflite_common 2.5.13+1
+ sqflite_darwin 2.4.4
```

```
final path = p.join(
  await getDatabasesPath(),
  'catatan.db',
);

_db = await openDatabase(
  path,
  version: 1,
  onCreate: (db, version) {
    return db.execute(
```

```
            'CREATE TABLE catatan('
            'id INTEGER PRIMARY KEY AUTOINCREMENT, '
            'judul TEXT NOT NULL, '
            'isi TEXT NOT NULL)'
          );
        },
      );
```

Tabel memiliki id sebagai primary key otomatis, serta judul dan isi sebagai data catatan. DbHelper menjadi lapisan yang menangani akses database. Bagian B membuat database catatan.db dan tabel catatan. Versi database pada tahap awal adalah 1. Hasilnya adalah:



5.3 Bagian C : Halaman Daftar Catatan

Halaman daftar mengambil data secara asynchronous menggunakan FutureBuilder. Data ditampilkan dengan ListView.builder dan dapat dibuka untuk diedit atau dihapus.

```
Future<void> _buka([Catatan? catatan]) async {
  await Navigator.push(
    context,
    MaterialPageRoute(
      builder: (_) => FormCatatanPage(catatan: catatan),
    ),
  );

  if (!mounted) return;
```

```
  _muat();
}
```

Hasil:



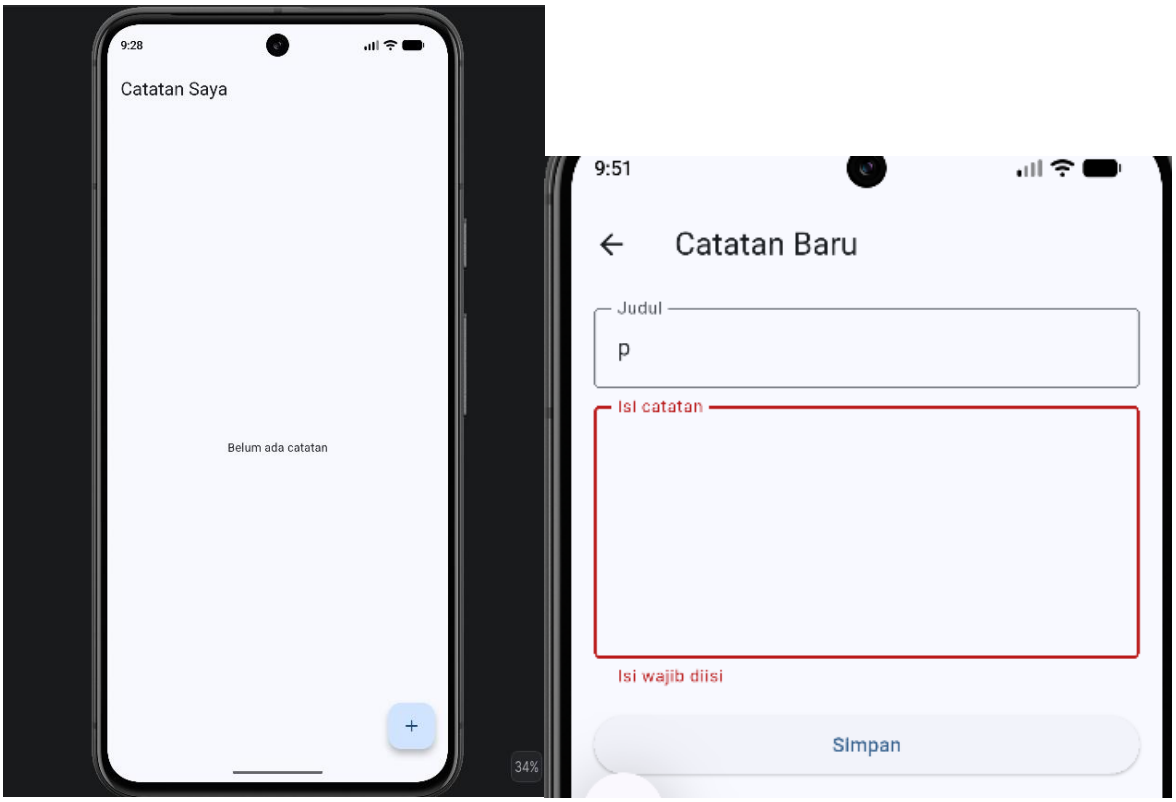
Setelah kembali dari form, _muat() dipanggil sehingga daftar mengambil data terbaru dari database.

5.4 Bagian D : Form Tambah dan Ubah

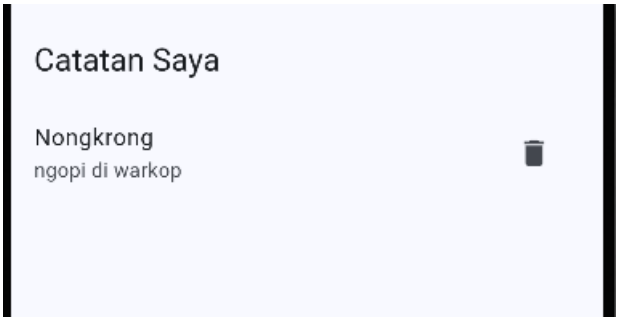
Bagian D menggunakan satu FormCatatanPage untuk dua fungsi. Jika catatan belum diberikan, form digunakan untuk menambah data. Jika catatan tersedia, form digunakan untuk mengubah data.

```
if (widget.catatan == null) {
  await DbHelper.tambah(c);
} else {
  await DbHelper.ubah(c);
}
```

Validasi dilakukan melalui FormState sehingga judul dan isi tidak boleh kosong.



Ketika catatan di simpan maka akan kembali ke halaman utama dan menampilkan catatan yang dibuat sebelumnya.



6. Latihan Mandiri

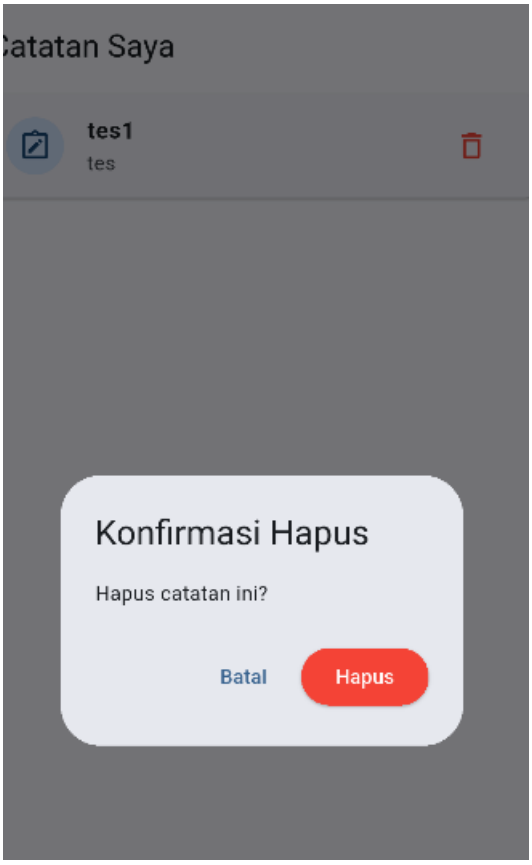
6.1 Latihan 1 : Konfirmasi Penghapusan

Sebelum penghapusan dijalankan, aplikasi menampilkan AlertDialog. Data hanya dihapus apabila pengguna memilih tombol Hapus.

```
final bool? konfirmasi = await showDialog<bool>(
  context: context,
  builder: (context) {
    return AlertDialog(
      title: const Text('Konfirmasi Hapus'),
```

```
      content: const Text('Hapus catatan ini?'),
      ...
    );
  },
);

if (konfirmasi != true) return;
await DbHelper.hapus(id);
```

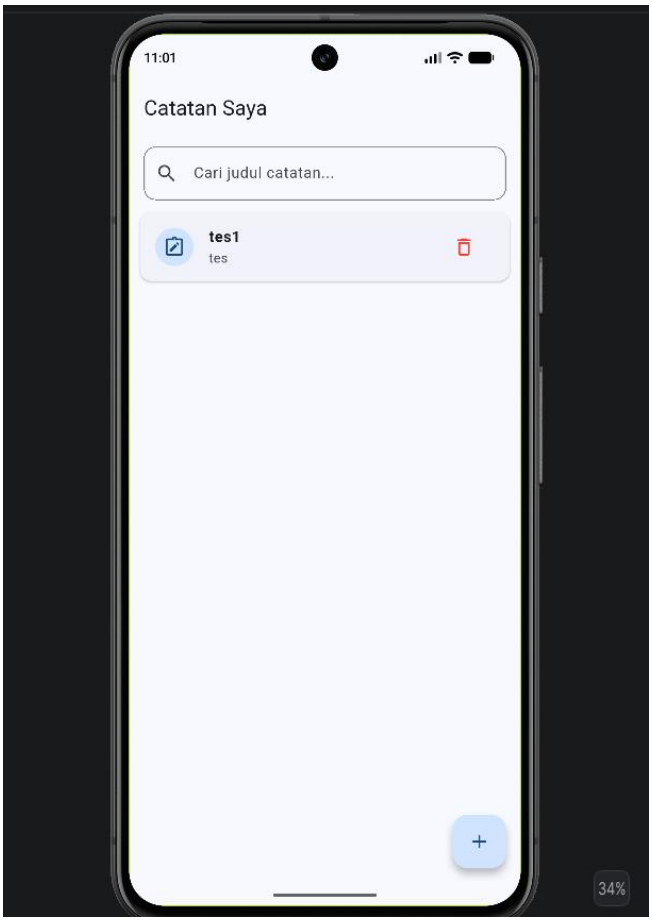


6.2 Latihan 2 : Pencarian Judul

Pencarian diterapkan berdasarkan kolom judul menggunakan operator LIKE. Parameter query diberikan melalui whereArgs sehingga kata pencarian menjadi parameter, bukan bagian langsung dari string SQL.

```
final rows = await db.query(
  'catatan',
  where: 'judul LIKE ?',
  whereArgs: ['%$kata%'],
  orderBy: 'id DESC',
);
```

Pada halaman daftar, perubahan teks pencarian memanggil _cariCatatan() kemudian _muat() untuk mengambil hasil pencarian terbaru.



6.3 Latihan 3 : Kolom Dibuat dan Migrasi Database

Latihan ketiga menambahkan kolom dibuat pada tabel catatan. Versi database dinaikkan dari 1 menjadi 2. Jika database lama masih menggunakan versi 1, onUpgrade menjalankan ALTER TABLE.

```
version: 2,

onUpgrade: (db, oldVersion, newVersion) async {
  if (oldVersion < 2) {
    await db.execute(
      'ALTER TABLE catatan ADD COLUMN dibuat TEXT',
    );
  }
}
```

Saat menambah struktur tabel pada database yang sudah pernah dibuat, kenaikan version diperlukan agar sqflite menjalankan proses migrasi. Tanpa kenaikan versi, onUpgrade tidak dipicu untuk perubahan tersebut.

```
dibuat: widget.catatan == null
    ? DateTime.now().toIso8601String()
    : widget.catatan!.dibuat,
```

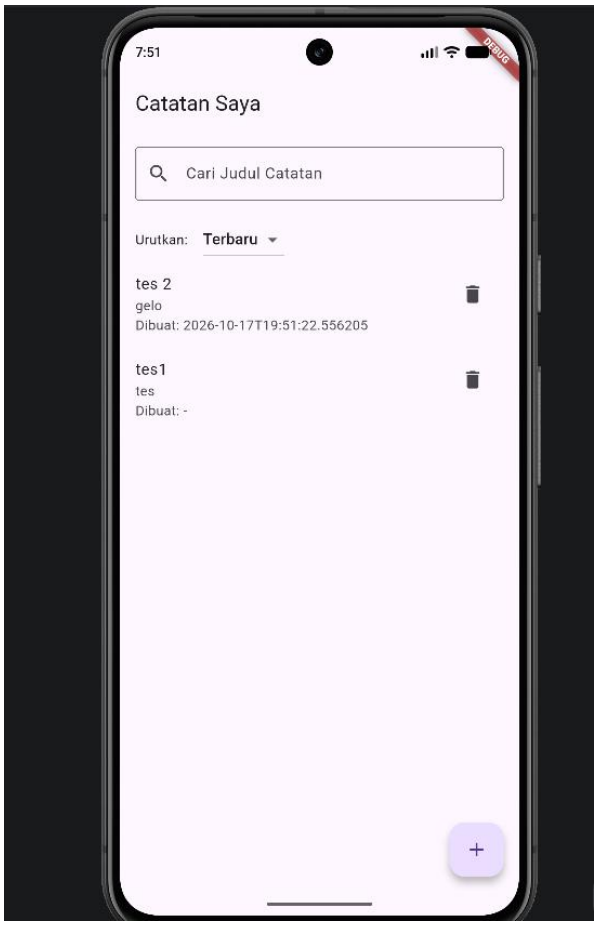
Tanggal dibuat disimpan sebagai teks ISO 8601 dan ditampilkan kembali dalam format tanggal dan waktu.

6.4 Latihan 4 : Pengurutan Terbaru/Terlama

Latihan keempat menambahkan pilihan urutan Terbaru atau Terlama. Pilihan tersebut disimpan dengan SharedPreferences sehingga preferensi tetap digunakan ketika aplikasi dibuka kembali.

```
orderBy: terbaru ? 'id DESC' : 'id ASC'
await prefs.setBool('urutanTerbaru', nilai);
```

Nilai true digunakan untuk urutan terbaru, sedangkan false digunakan untuk urutan terlama, Implementasi lengkap juga mempertahankan pencarian ketika urutan berubah.



7. Tugas

7.1 Deskripsi

Tugas mengembangkan aplikasi Pencatat Pengeluaran menggunakan SQLite sebagai penyimpanan utama dan SharedPreferences untuk menyimpan satu pengaturan aplikasi.

7.2 Struktur Data

Field	Tipe	Keterangan
id	Integer	Primary key dan auto increment
nama	Text	Nama pengeluaran
jumlah	Integer	Nominal pengeluaran
kategori	Text	Kategori pengeluaran
tanggal	Text	Tanggal pengeluaran

7.3 Model Pengeluaran

```
class Pengeluaran {
  final int? id;
  final String nama;
  final int jumlah;
  final String kategori;
  final String tanggal;
}
```

Model memetakan setiap baris tabel pengeluaran menjadi objek Dart. toMap() digunakan ketika data akan dimasukkan atau diperbarui, sedangkan fromMap() digunakan ketika data dibaca dari SQLite.

7.4 Database SQLite

```
'CREATE TABLE pengeluaran('
'id INTEGER PRIMARY KEY AUTOINCREMENT, '
'nama TEXT NOT NULL, '
'jumlah INTEGER NOT NULL, '
'kategori TEXT NOT NULL, '
'tanggal TEXT NOT NULL)'
```

DbHelper menyediakan fungsi tambah, semua, ubah, dan hapus yang merepresentasikan operasi CRUD.

7.5 Halaman Daftar dan Total

Halaman utama mengambil data melalui FutureBuilder. Total pengeluaran dihitung menggunakan fold terhadap seluruh data yang berhasil dibaca.

```
final total = data.fold<int>(
  0,
  (jumlah, item) => jumlah + item.jumlah,
);
```

Setiap item menampilkan nama, kategori, tanggal, nominal, dan tombol hapus. Menekan item membuka form untuk melakukan perubahan.

7.6 Form Pengeluaran dan Validasi

Form memiliki input nama, jumlah, dan kategori. Jumlah harus berupa angka dan lebih besar dari nol, sedangkan kategori harus dipilih.

```
final angka = int.tryParse(v.trim());

if (angka == null) {
  return 'Jumlah harus berupa angka';
}

if (angka <= 0) {
  return 'Jumlah harus lebih dari 0';
}
```

Kategori dibuat menggunakan DropdownButtonFormField dengan pilihan Makanan, Transportasi, Belanja, Lainnya.

7.7 SharedPreferences pada Tugas

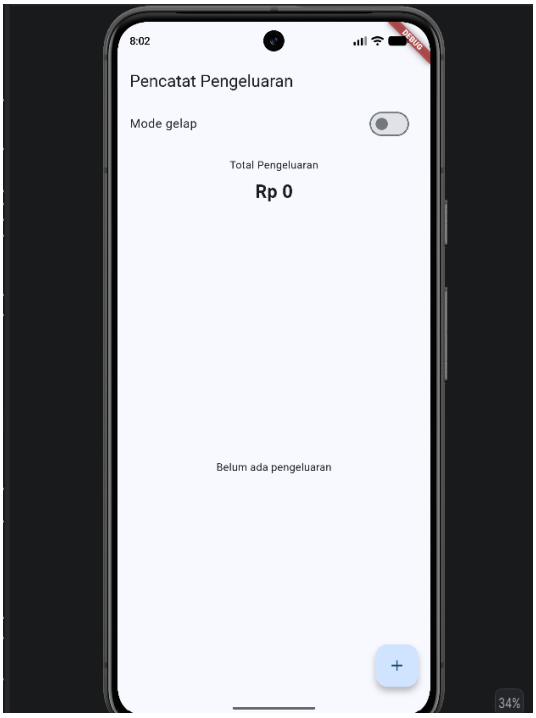
```
final prefs = await SharedPreferences.getInstance();

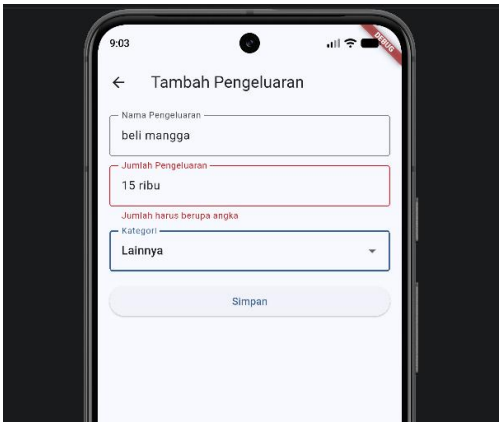
_gelap = prefs.getBool('modeGelap') ?? false;

await prefs.setBool('modeGelap', nilai);
```

SharedPreferences digunakan untuk mempertahankan pilihan mode gelap. Ketika aplikasi dibuka kembali, nilai mode dibaca dan diterapkan pada ThemeData.

7.8 Hasil


8:02
Pencatat Pengeluaran
Mode gelap
Total Pengeluaran
Rp 0
Belum ada pengeluaran
34%


9:03
Tambah Pengeluaran
Nama Pengeluaran
beli mangga
Jumlah Pengeluaran
15 ribu
Jumlah harus berupa angka
Kategori
Lainnya
Simpan


9:04
Tambah Pengeluaran
Nama Pengeluaran
w
Jumlah Pengeluaran
Jumlah wajib diisi
Kategori
Belanja
Simpan

8. Pengujian

No	Skenario	Hasil yang Diharapkan	Status
1	Menyimpan nama pada Bagian A	Nama tersimpan dan dapat dibaca kembali	Berhasil
2	Mengaktifkan mode gelap	Tema berubah dan preferensi tersimpan	Berhasil
3	Membuat database	Database dan tabel catatan tersedia	Berhasil
4	Menambah catatan	Catatan muncul pada daftar	Berhasil
5	Mengubah catatan	Isi catatan diperbarui	Berhasil
6	Menghapus catatan	Catatan terhapus	Berhasil
7	Menghapus catatan lalu batal	Data tetap ada	Berhasil
8	Mencari judul	Hanya catatan yang sesuai kata ditampilkan	Berhasil
9	Mengubah urutan	Daftar berubah menjadi terbaru/terlama	Berhasil
10	Migrasi database	Kolom dibuat tersedia setelah version dinaikkan	Berhasil
11	Menambah pengeluaran valid	Data pengeluaran tersimpan	Berhasil
12	Jumlah kosong/bukan angka/≤ 0	Pesan validasi ditampilkan	Berhasil
13	Kategori tidak dipilih	Pesan validasi ditampilkan	Berhasil
14	Mengubah dan menghapus pengeluaran	Data diperbarui/terhapus	Berhasil
15	Restart aplikasi	Data lokal dan preferensi tetap tersedia	Berhasil

9. Hasil dan Pembahasan

Berdasarkan implementasi yang dikerjakan, SharedPreferences berhasil digunakan untuk menyimpan preferensi sederhana, sedangkan SQLite digunakan untuk data yang memiliki struktur tabel. Aplikasi catatan telah dikembangkan dari CRUD dasar menjadi aplikasi dengan pencarian, konfirmasi penghapusan, tanggal dibuat, dan pilihan urutan. Pada tugas, pola yang sama diterapkan pada data pengeluaran dengan tambahan validasi jumlah, kategori, perhitungan total, serta penyimpanan mode gelap.

Pada implementasi latihan ketiga, database menggunakan version 2 dan onUpgrade menambahkan kolom dibuat. Hal ini menunjukkan pentingnya version database dalam menjaga perubahan struktur database yang sudah ada.

10. Kesimpulan

Praktikum Pertemuan 5 memberikan pemahaman mengenai penyimpanan lokal pada Flutter menggunakan SharedPreferences dan SQLite. SharedPreferences sesuai untuk menyimpan preferensi sederhana berbentuk key-value, sedangkan SQLite sesuai untuk data terstruktur dan operasi CRUD. Implementasi aplikasi catatan menunjukkan penggunaan database lokal, validasi, pencarian, migrasi, dan pengurutan. Konsep tersebut kemudian diterapkan pada aplikasi Pencatat Pengeluaran sehingga data dapat dikelola secara lokal dan tetap tersedia setelah aplikasi dijalankan kembali.

11. Refleksi

Mengapa SharedPreferences kurang cocok untuk menyimpan ratusan catatan?

Karena SharedPreferences ditujukan untuk pasangan key-value sederhana dan tidak menyediakan kemampuan query, relasi, serta pengelolaan data terstruktur seperti SQLite.

Mengapa query menggunakan tanda ? dan whereArgs?

Agar nilai pencarian diberikan sebagai parameter query. Pendekatan ini membuat pemisahan antara struktur SQL dan nilai yang dicari menjadi lebih aman dan sesuai dengan API sqflite.

Mengapa daftar perlu dimuat ulang setelah kembali dari form?

Karena operasi tambah atau ubah terjadi di halaman form. Setelah kembali, halaman daftar perlu mengambil data terbaru dari database agar tampilan sesuai dengan kondisi penyimpanan.

Apa fungsi onCreate dan onUpgrade?

onCreate digunakan ketika database dibuat pertama kali. onUpgrade digunakan ketika versi database dinaikkan sehingga perubahan struktur database, seperti penambahan kolom, dapat diterapkan pada database yang sudah ada.